

PES UNIVERSITY, BENGALURU

Department of Computer Science and Engineering

Software Architecture and Design Specification

Online Bookstore

Version 1.1 – Final (synchronized with SRS v1.1 Final)

Based on SRS v1.1 Final, Software Test Plan v1.1 and Security & Integration Plan v1.1

Prepared by: Online Bookstore Project Team

M1 – Pooja Avadhani | M2 – Rahul M S | M3 – Preksha B | M4 – Ojas Binjola

Date: October 2026

Status: Final

Revision History

Version	Date	Author	Change Summary
1.0	October 2026	Project team	Initial SAD draft based on the supplied SRS and template.
1.1 (draft)	October 2026	Project team	Synchronized with SRS requirement IDs, Test Plan test cases, Security & Integration Plan, data model, integration contracts, security controls, route set and consistency findings.
1.1 (Final)	October 2026	Project team	Brought to the SRS v1.1 Final baseline: all previously proposed or open decisions recorded as resolved; SO-1 to SO-6 and SEC-CTRL-01 to 20 aligned with the SRS wording; Book now records its Seller; FR-AUTH-03 and FR-CART-03 added to traceability; test-case references changed to the STP IDs (TC-01 to TC-27); integration scenario IT-02 corrected (listing price editing is out of scope); use-case model added (Section 3.11).

Approvals

Role	Name	Signature / Date
Project representative	Preksha	Pending
Instructor / guide	Ashok Kumar Patil	Pending

Note: The SRS v1.1 Final is the requirements authority. Where this document describes design choices that go beyond the SRS (module layout, route names, library choices), they are implementation guidance and are marked as such.

Contents

- 1 Introduction
- 2 Document Overview
- 3 Architecture
 - 3.1 Goals and Constraints
 - 3.2 Stakeholders and Concerns
 - 3.3 Component (UML) Diagram
 - 3.4 Component Descriptions
 - 3.5 Chosen Architecture Pattern and Rationale
 - 3.6 Data Model
 - 3.7 Technology Stack and Data Stores
 - 3.8 Risks and Mitigations
 - 3.9 Traceability to Requirements
 - 3.10 Security Architecture
 - 3.11 Use-Case Model
- 4 Design
 - 4.1 Design Overview
 - 4.2 UML Sequence Diagrams
 - 4.3 API Design
 - 4.4 Error Handling, Logging and Monitoring
 - 4.5 UX Design
 - 4.6 Integration Contracts and Build Order
 - 4.7 Resolved Decisions and Remaining Open Items
- 5 Appendices
 - Document Review Checklist

1 Introduction

1.1 Purpose

This Software Architecture and Design (SAD) document defines the architecture, components, data model, interactions, interfaces, security controls, integration contracts and verification traceability for the Online Bookstore. It translates the requirements in the Online Bookstore Software Requirements Specification, Version 1.1 Final (SRS) into a design that can be implemented and tested.

The design is aligned to the SRS, the Software Test Plan (STP) and the Security & Integration Plan. The SRS is the authoritative baseline; where this document adds implementation detail it says so.

1.2 Scope

The design covers user registration, login and logout; book browsing and searching; seller book-listing creation; customer cart add/remove/view operations; order placement; order-history viewing; role-based access control; and protection of user data.

Payment processing, shipping and delivery tracking, reviews and ratings, recommendations, wishlists, discounts, inventory replenishment, seller payouts, editing or deleting listings, administrator dashboards and mobile applications are outside the current scope unless formally approved and added to the SRS.

1.3 Audience

- Developers implementing the Django application, models, views, forms and templates.
- Testers preparing requirement-based, integration, authorization, regression, performance, usability and compatibility tests.
- Project guide or reviewer assessing architecture quality, security and requirement coverage.
- Team members maintaining the design as implementation proceeds.

1.4 Definitions

Term	Definition
SAD	Software Architecture and Design.
SRS	Software Requirements Specification.
STP	Software Test Plan.
Django ORM	Django's model-based interface for database operations.
RBAC	Role-Based Access Control; restricting operations according to an assigned role.
PK / FK	Primary key / foreign key.
CSRF	Cross-Site Request Forgery.
XSS	Cross-Site Scripting.
Cart item	A selected book and quantity associated with a customer.
Order item	A line in an order containing a book, quantity and purchase-time unit price.
IDOR	Insecure Direct Object Reference; unauthorized access to another user's object by changing an identifier.
UAT	User Acceptance Testing.

1.5 Related Documents

- Online Bookstore Software Requirements Specification, Version 1.1 – Final, October 2026.
- Online Bookstore Software Test Plan, Version 1.1, October 2026.

- Online Bookstore – Security & Integration Plan (M4), Version 1.1, October 2026.
- Software Architecture and Design Specification template supplied for the project.

2 Document Overview

2.1 How to Use This Document

Section 3 defines the architecture, components, data model, technology choices, risks, requirement traceability, security architecture and use-case model. Section 4 defines implementation-level design, workflows, routes, validation, error handling, UX, integration contracts and the decision log. Section 5 contains the glossary, references, tools and the final review checklist. Testing is defined in the Software Test Plan; Section 3.9 maps every requirement to its test cases.

2.2 Related Design Status

All decisions that were open in earlier drafts have been resolved in SRS v1.1 Final and are recorded below as fact. Only the deployment environment remains open.

Decision	Status	Design treatment
Application framework	Resolved (SRS 2.4, 3.8.2)	Python Django is mandatory.
Database engine	Resolved (SRS 2.1, 3.1.2)	Django ORM; SQLite for development and demonstration, PostgreSQL if hosted.
Deployment environment	Open (SRS Appendix D)	Logical browser–Django–database deployment is defined; hosting is chosen by the team before the demonstration.
Registration role selection	Resolved (FR-AUTH-01, BR-04)	The user chooses Customer or Seller; the server validates the value; the role cannot be changed afterwards.
Seller purchasing	Resolved (BR-03)	A Seller cannot purchase in this version; purchasing operations are Customer-only.
Security objectives	Resolved (SRS 3.5.1)	SO-1 to SO-6 are approved in the SRS and used unchanged in Section 3.10.1.
Detailed security controls	Resolved (SRS 3.5.4)	SEC-CTRL-01 to SEC-CTRL-20 under SEC-01 and SEC-02. SEC-CTRL-16 is a recommended, optional enhancement.
Book ownership	Resolved (FR-BOOK-03)	Book stores its creating Seller (seller_id). Editing and deleting listings are out of scope.
Duplicate add-to-cart	Resolved (FR-CART-01, BR-08)	Increase the quantity of the existing line; unique (customer, book); maximum 10.
Use-case diagram	Approved (SRS Figure 1)	Reproduced in Section 3.11.

3 Architecture

3.1 Goals and Constraints

Goals

- Cover every functional and non-functional requirement in SRS v1.1.
- Separate authentication, book management, cart and order responsibilities.
- Maintain cart and order data consistency after successful operations.
- Meet the SRS search response-time target of under three seconds under normal operating conditions.

- Support the latest stable versions of Google Chrome, Mozilla Firefox and Microsoft Edge.
- Keep requirements independently testable and preserve clear module boundaries.
- Enforce security at the server boundary rather than relying on interface controls.

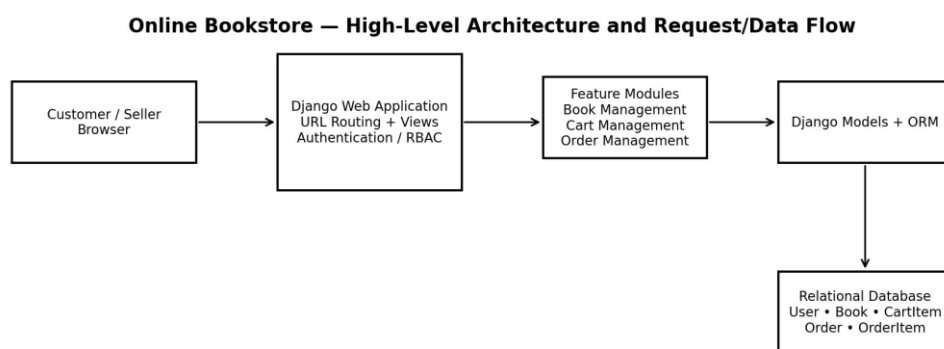
Constraints

- The application shall use Python Django and operate as a web-based application.
- The system shall persist user accounts, roles, book listings, cart data and orders in a relational database (SQLite for development and demonstration; PostgreSQL if hosted).
- Customer and Seller are the two registered roles; Anonymous Visitor is an unauthenticated user.
- Protected operations must enforce role restrictions on the server.
- User data must be protected from unauthorized access and disclosure.
- The SRS availability target is at least 99% of scheduled operating time, excluding planned maintenance.
- The deployment environment is not yet fixed.

3.2 Stakeholders and Concerns

Stakeholder	Primary concerns	Architectural response
Customer	Simple browsing/search, correct cart and orders, privacy.	Clear pages, validation, user-scoped cart/order queries, server-side authorization.
Seller	Straightforward book-listing creation and access to catalogue/search.	Seller-only listing route with server-side role check; catalogue/search remains available.
Developers	Modularity, maintainability, safe integration.	Layered modular monolith; explicit module contracts and Django ORM.
Testers	Verifiable requirements and repeatable tests.	Exact SRS IDs, test-case traceability, validation rules and explicit authorization paths.
Project guide/reviewer	Consistency across SRS, SAD, STP, security plan and diagrams.	Single naming scheme, requirement IDs, traceability and decision log.
Deployment operator	Availability, secrets, backups, safe configuration.	Environment-specific settings, HTTPS, secure cookies, backups, recovery checks.

3.3 Component (UML) Diagram



Browser input is untrusted; authentication, authorization and validation occur on the server.

Figure 1. High-level architecture and request/data flow.

The browser is treated as an untrusted boundary. Django receives requests, authenticates the user, enforces the RBAC rule, validates input, coordinates feature logic and accesses persistent data through the Django ORM. Business values such as prices, totals and ownership are recomputed on the server.

3.4 Component Descriptions

Component	Responsibility
Browser / User Interface	Registration, login, logout, catalogue, search, cart, order and seller-listing pages; submits user actions.
URL Routing and Views	Routes requests; checks authentication and role; validates forms; calls feature logic; returns pages or redirects.
Accounts / Authentication	Registration, login/logout, sessions, password hashing and server-side role handling.
Book Management	Catalogue browsing/search and seller book-listing creation (recording the creating Seller).
Cart Management	Add, view and remove cart items; validates quantity and ownership.
Order Management	Validates cart, reads current book prices, calculates totals, creates Order and OrderItem records, clears cart on successful order and returns history.
Django Models and ORM	Represents persistent entities and performs parameterized database operations.
Relational Database	Stores users, roles, books, cart items, orders and order items with constraints.

3.5 Chosen Architecture Pattern and Rationale

Chosen pattern: layered modular monolith implemented with Django.

- Presentation layer: templates, forms, static assets and browser interaction.
- Request/access layer: URL routing and views for authentication, authorization, validation and request handling.
- Feature layer: focused modules/service functions for Book Management, Cart Management and Order Management.
- Data-access layer: Django models and ORM.
- Persistence layer: relational database.

This structure fits the bounded scope and the mandatory Django framework. Microservices are not justified by the current requirements and would add unnecessary deployment and debugging complexity.

3.6 Data Model

The logical model below is identical to SRS Figure 2 and Appendix B. Book records the Seller who created it; OrderItem preserves the purchase-time unit price; CartItem is unique per (customer, book). Exact Django field names are chosen at implementation but must preserve these entities, relationships and constraints.

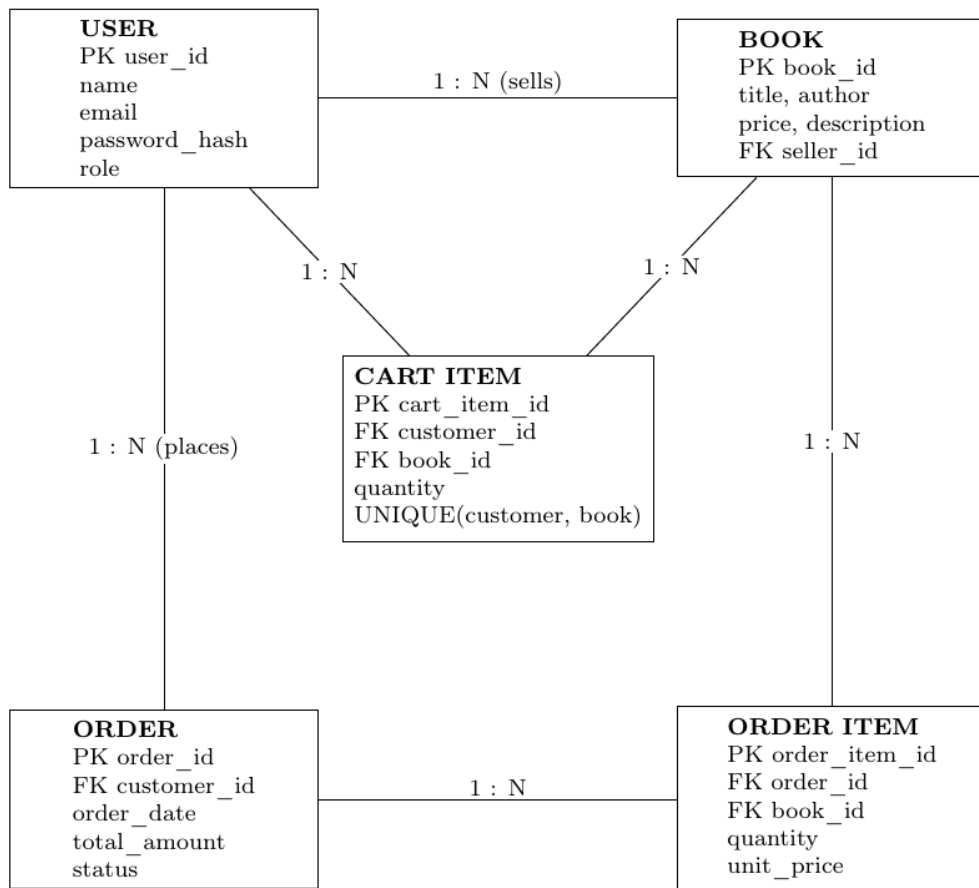


Figure 2. Logical data model (identical to SRS Figure 2).

Entity	Key fields / constraints
User	user_id (PK), name, email (unique), password hash, role ∈ {Customer, Seller}. Role is stored server-side and set at registration.
Book	book_id (PK), title, author, price (positive, two decimals), description (optional), seller_id (FK to User, set by the system).
CartItem	cart_item_id (PK), customer_id (FK), book_id (FK), quantity 1 to 10. UNIQUE(customer_id, book_id). Price is not copied into the cart.
Order	order_id (PK), customer_id (FK), order_date, total_amount, status ("Placed" on creation).
OrderItem	order_item_id (PK), order_id (FK), book_id (FK), quantity, unit_price captured at purchase time.

3.7 Technology Stack and Data Stores

Area	Technology / approach	Status
Web framework	Python Django	Required by SRS.
Data access	Django ORM and Django models	Required (SEC-CTRL-14).
Database	SQLite for development and demonstration; PostgreSQL if hosted	Decided (SRS 2.1, 3.1.2).
User interface	Django templates, HTML, CSS (optional JavaScript)	Required templates/HTML/CSS (SRS 3.8.2).
Authentication	Django authentication and password hashing	Required (SEC-CTRL-01, 04).
Transport security	HTTPS in hosted environments; secure cookies	Required when hosted (SEC-CTRL-17).
Version control	Git	Recommended.

Area	Technology / approach	Status
Secrets/configuration	Environment variables for hosted secrets; DEBUG=False	Required when hosted (SEC-CTRL-17).

3.8 Risks and Mitigations

Risk	Impact	Mitigation
Hosted database differs from development database	Migration or environment rework.	Keep models portable; test migrations on PostgreSQL before hosting.
Incorrect role enforcement	Unauthorized listing/purchasing actions.	Enforce role server-side on every protected route; run positive and negative tests.
Client tampering with prices/quantities	Incorrect totals or invalid data.	Read prices from stored Book records; validate quantities server-side.
Concurrent order/cart changes	Inconsistent order or cart state.	Use transaction.atomic() for order creation; clear cart only inside the successful transaction.
IDOR / ownership bypass	User can access another user's cart/order.	Scope queries to request.user and return 404 when the object is not owned.
Brute-force login	Account compromise or resource abuse.	Django password validators (mandatory); optional login throttling (SEC-CTRL-16, e.g. django-axes).
Sensitive information in logs/repository	Credential or privacy exposure.	Never log passwords, session IDs, tokens or secrets; keep secrets out of source control.
Host/database outage	Service unavailable.	Choose hosting appropriate to the 99% target; backups and restore checks.
Search abuse	Slow or overloaded search.	Limit query length to 100 characters, paginate at 20, measure response time, add indexes where evidence supports them.
Open redirect/session misuse	Phishing or session abuse.	Regenerate session on login, destroy at logout, validate next redirect, secure/HttpOnly cookies.

3.9 Traceability to Requirements

The matrix uses the exact SRS requirement IDs and the STP test-case IDs (TC-01 to TC-27) and integration test IDs (IT-01 to IT-06). It is identical in content to SRS Appendix C.1.

SRS ID	Requirement	Architecture / design mapping	STP test cases	Integration
FR-AUTH-01	User registration	Accounts / Authentication; §4.2.1, §4.3	TC-01, TC-02	–
FR-AUTH-02	User login	Accounts / Authentication; §4.2.1, §4.3	TC-03, TC-04	IT-06
FR-AUTH-03	User logout	Accounts / Authentication; §4.3 (/logout/)	TC-25	–
FR-BOOK-01	Browse books	Book Management; §4.3	TC-05	IT-01
FR-BOOK-02	Search books	Book Management; §4.3	TC-06, TC-07	IT-01
FR-BOOK-03	Seller adds book	Book Management; RBAC §3.10.3; §4.3	TC-08, TC-09	IT-01
FR-CART-01	Add book to cart	Cart Management; §4.2.2, §4.3	TC-10, TC-27	IT-02
FR-CART-02	Remove book from cart	Cart Management; §4.3	TC-11	IT-05
FR-CART-03	View cart	Cart Management; §4.3 (/cart/)	TC-26	IT-05
FR-ORDER-01	Place order	Order Management; transaction design; §4.2.3, §4.3	TC-12, TC-13	IT-02, IT-03, IT-04, IT-06
FR-ORDER-02	View order history	Order Management; §4.3	TC-14, TC-15	IT-03, IT-05
NFR-01	Search within 3 seconds	Book Management; §3.8, §4.4	TC-16	–

SRS ID	Requirement	Architecture / design mapping	STP test cases	Integration
NFR-02	Clear/consistent navigation	UX design; §4.5	TC-17	–
NFR-03	Cart/order consistency	Order transaction design; §3.6, §4.2.3	TC-18	IT-04
NFR-04	99% scheduled availability	Deployment/operations; §4.4	TC-19	–
NFR-05	Chrome/Firefox/Edge compatibility	Browser UI; §4.5	TC-20	–
NFR-06	Modular maintainability	Layered modular monolith; §3.5, §4.1	TC-21	–
NFR-07	Independent testability	Module boundaries; §4.1	TC-22	–
SEC-01	Role-based access	Security architecture; §3.10.3	TC-09, TC-23	IT-05
SEC-02	Protection of user data	Security architecture; §3.10.2	TC-15, TC-24	IT-05

3.10 Security Architecture

Security is enforced at the server boundary and data layer. Hiding a button in the UI is not access control. Every protected operation must authenticate the request, enforce the assigned role where required, validate input and scope data access to the logged-in user.

3.10.1 Security Objectives

SO-1 to SO-6 are approved and defined in SRS Section 3.5.1.

ID	Objective	Meaning for this system
SO-1	Confidentiality	A user can see only their own cart, orders and account data. Passwords are never stored in recoverable form.
SO-2	Integrity	Prices, totals, roles, ownership and order state are decided by the server and never trusted from the client. Orders are all-or-nothing.
SO-3	Availability	Misuse such as expensive search requests shall not take the site down; supports the 99% target of NFR-04.
SO-4	Authenticity	Every protected action is tied to a verified, logged-in identity.
SO-5	Authorization	Customers and Sellers can perform only the operations permitted for their role, enforced on the server.
SO-6	Accountability	Failed logins, denied access and failed orders are logged, without secrets, so incidents can be investigated.

3.10.2 Security Requirements and Controls

SEC-01 (role-based access) and SEC-02 (protection of user data) are the top-level SRS security requirements. The detailed controls below are SEC-CTRL-01 to SEC-CTRL-20, defined in SRS Section 3.5.4 and used here with the same identifiers and wording. Each control is verified in STP Section 16.2 under the same identifier. SEC-CTRL-16 is a recommended, optional enhancement and is not part of the mandatory acceptance baseline.

Control	Design rule	Obj.	Parent
SEC-CTRL-01	Store passwords only as salted hashes using Django's password hasher.	SO-1	SEC-02
SEC-CTRL-02	Enable password validators: minimum length 8, not a commonly used password, not entirely numeric.	SO-4	SEC-02
SEC-CTRL-03	Use one generic login failure message that does not reveal whether the email exists.	SO-1	SEC-02

Control	Design rule	Obj.	Parent
SEC-CTRL-04	Create a session only after valid credentials, regenerate it at login, destroy it at logout; an old session is invalid after logout.	SO-4	SEC-02
SEC-CTRL-05	Every protected route requires authentication; anonymous users are redirected to login.	SO-4	SEC-01
SEC-CTRL-06	Every protected route enforces the user's role on the server; a wrong role receives HTTP 403 with no data change.	SO-5	SEC-01
SEC-CTRL-07	Store the role server-side from the allowed set (Customer, Seller); never take it from a hidden field or URL; no form can make a user staff or administrator.	SO-5	SEC-01
SEC-CTRL-08	Scope cart/order queries to the logged-in user; ignore client-supplied customer IDs; another user's object returns HTTP 404.	SO-1	SEC-02
SEC-CTRL-09	Compute prices and totals on the server from stored Book records; ignore client-submitted prices.	SO-2	SEC-02
SEC-CTRL-10	Validate quantities as positive integers not exceeding 10 per cart line.	SO-2	SEC-02
SEC-CTRL-11	Create Order, OrderItems and clear the cart inside one database transaction.	SO-2	SEC-02
SEC-CTRL-12	All state-changing requests use POST with a valid CSRF token.	SO-2	SEC-02
SEC-CTRL-13	Output-escape user-supplied content (book title, description, search text); no raw HTML rendering.	SO-2	SEC-02
SEC-CTRL-14	Use the Django ORM only; no SQL built from user input.	SO-2	SEC-02
SEC-CTRL-15	Limit search input to 100 characters and paginate results at 20 per page.	SO-3	SEC-02
SEC-CTRL-16	Recommended enhancement (optional): throttle repeated failed logins (e.g. 5 consecutive attempts) with a temporary lockout. Library option: django-axes.	SO-3, SO-4	SEC-02
SEC-CTRL-17	In a hosted environment: DEBUG=False, HTTPS, secure and HttpOnly cookies, HSTS, secrets from environment variables.	SO-1	SEC-02
SEC-CTRL-18	Never log passwords, session identifiers, secret keys or tokens; log failed login, denied access and failed order events with user identifier and timestamp.	SO-6	SEC-02
SEC-CTRL-19	Show generic user-facing error pages; never expose stack traces.	SO-1	SEC-02
SEC-CTRL-20	Validate the post-login next/return address against the application host; ignore external addresses.	SO-4	SEC-02

3.10.3 RBAC Matrix

Operation / route	Anonymous Visitor	Customer	Seller
Register	Allowed	Not Allowed	Not Allowed
Login	Allowed	Allowed	Allowed
Logout	Not Allowed	Allowed	Allowed
Browse / Search (/books/, /books/search/)	Allowed	Allowed	Allowed
Add listing (/books/add/)	Login required	403 Denied	Allowed
View cart (/cart/)	Login required	Own cart only	403 Denied
Add / remove cart item	Login required	Own cart only	403 Denied
Place order (/orders/place/)	Login required	Own cart only	403 Denied
View order history (/orders/)	Login required	Own orders only	403 Denied

This matrix is identical to SRS Section 3.5.5 and is the single source of truth for route-level role behaviour. Purchasing is Customer-only (BR-03).

3.11 Use-Case Model

The use-case diagram below is the approved SRS Figure 1. Every use case maps to one SRS functional requirement and one route in Section 4.3.

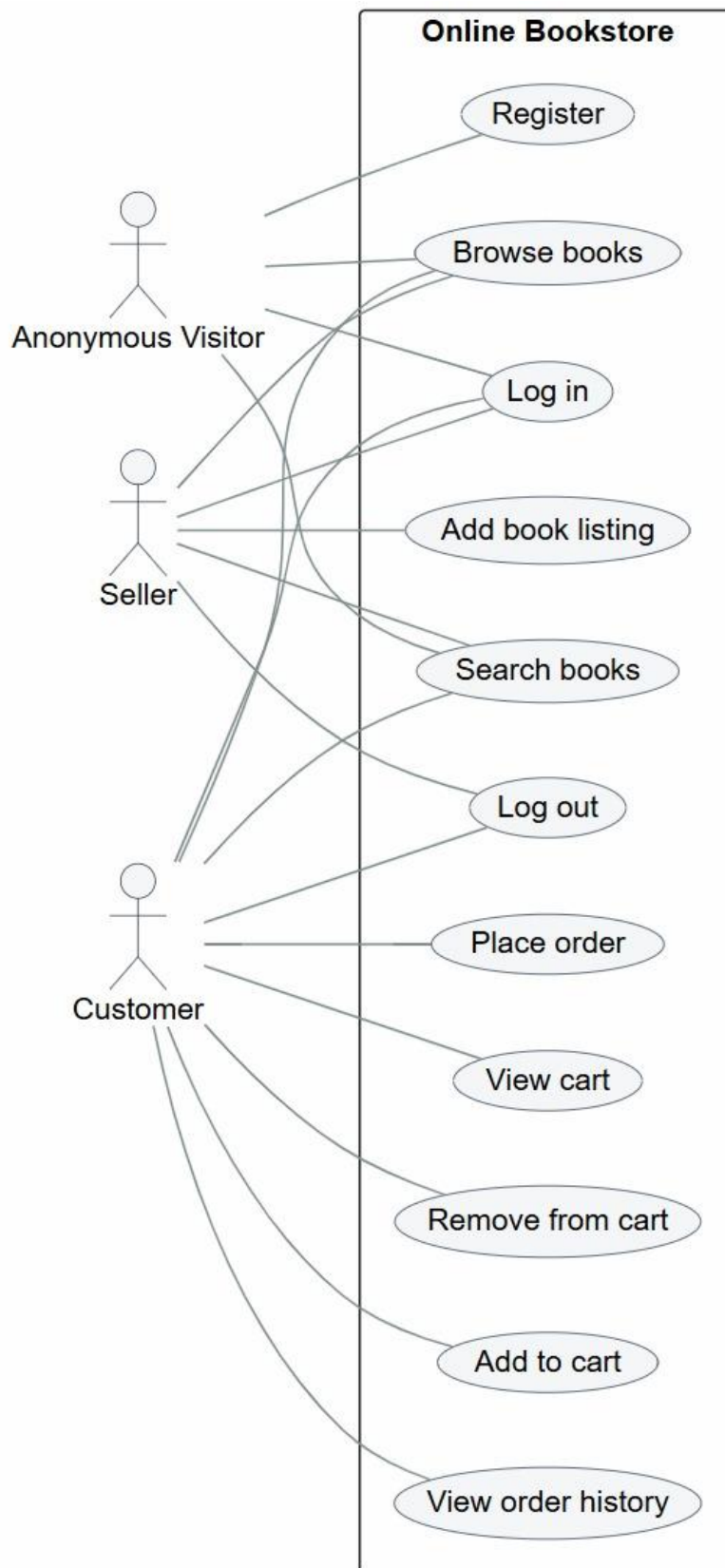


Figure 3. Online Bookstore use-case diagram (identical to SRS Figure 1).

Use case	Actors	Requirement	Route
Register	Anonymous Visitor	FR-AUTH-01	/register/
Log in	Anonymous Visitor, Customer, Seller	FR-AUTH-02	/login/
Log out	Customer, Seller	FR-AUTH-03	/logout/
Browse books	Anonymous Visitor, Customer, Seller	FR-BOOK-01	/books/
Search books	Anonymous Visitor, Customer, Seller	FR-BOOK-02	/books/search/
Add book listing	Seller	FR-BOOK-03	/books/add/
Add to cart	Customer	FR-CART-01	/cart/add/
Remove from cart	Customer	FR-CART-02	/cart/remove/<id>/
View cart	Customer	FR-CART-03	/cart/
Place order	Customer	FR-ORDER-01	/orders/place/
View order history	Customer	FR-ORDER-02	/orders/

4 Design

4.1 Design Overview

Django URL routing dispatches requests to focused views. Views check authentication and role, validate forms, call feature logic and return templates or redirects. Models represent persistent entities and the ORM handles database access. Operations that modify multiple related records, especially order placement, use database transactions.

Suggested Django project organization (module layout is implementation guidance; the module responsibilities and contracts in Section 4.6 are binding).

Module / path	Responsibility
config/ or project package	Settings, root URLs, WSGI/ASGI entry points.
accounts/	Registration, login/logout, role assignment, account forms, authentication helpers and the role_required decorator.
books/	Catalogue browsing/search and seller listing creation (Book model with seller FK).
cart/	CartItem model and add/remove/view operations; ownership checks.
orders/	Order and OrderItem models, order placement, transaction handling, order history.
templates/ and static/	HTML templates, CSS, images and JavaScript assets.
tests/	Model, form, view, permission, workflow, integration and regression tests.
requirements.txt / environment config	Python dependencies and deployment-specific configuration; no secrets in source.

4.2 UML Sequence Diagrams

4.2.1 Registration and Login

1. The user opens the registration or login page.
2. The browser submits the form to Django.
3. Django validates the submitted fields and credentials.
4. For registration, the system creates the account only after validation; the Role must be Customer or Seller and is validated by the server (SEC-CTRL-07).

- For login, Django establishes a session only when credentials are valid and regenerates the session (SEC-CTRL-04).
- The system redirects to the appropriate page or displays a safe error.

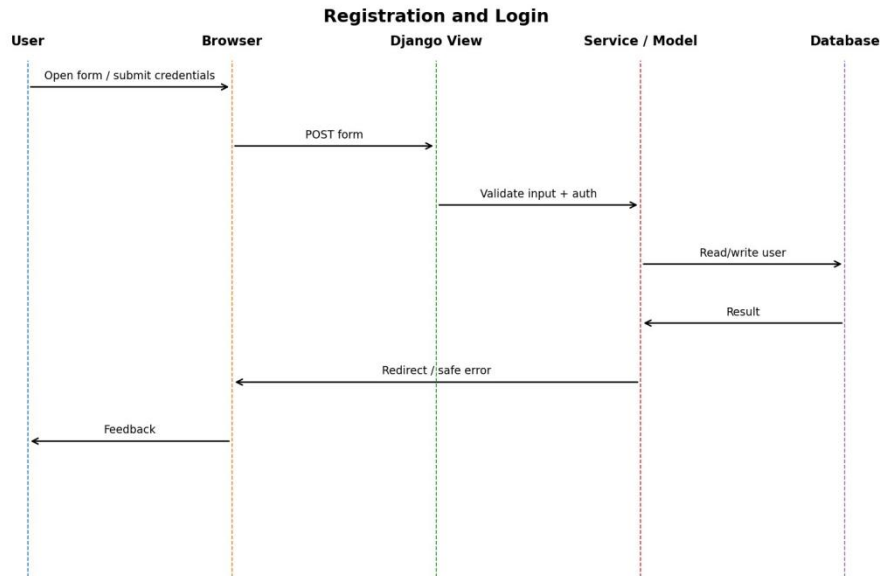


Figure 4. Registration and login interaction sequence.

Logout (FR-AUTH-03) is a POST to /logout/ that destroys the current session and redirects safely; a request reusing the old session is treated as made by an Anonymous Visitor.

4.2.2 Add a Book to Cart

- The Customer submits the add-to-cart form.
- Django checks the authenticated session and Customer role.
- The application verifies that the book exists and the quantity is a positive integer not exceeding 10.
- Cart logic creates a CartItem or, if the book is already in the cart, increases the quantity of the existing line (BR-08); the (customer, book) pair is unique.
- The database persists the change and the interface shows the updated cart or a safe error.

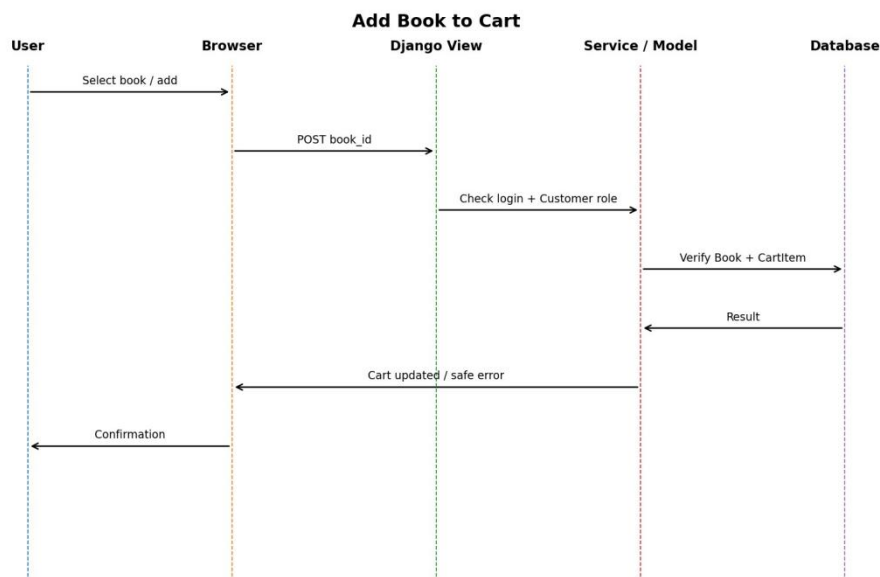


Figure 5. Add-to-cart interaction sequence.

4.2.3 Place an Order

1. The Customer submits order placement from the cart.
2. Django checks login status, Customer role and cart contents; an empty cart creates no order.
3. The server loads Book records and current prices, validates quantities/items and calculates the total.
4. Within one database transaction, the application creates the Order (status “Placed”) and associated OrderItem records, copying current unit prices into OrderItem.unit_price.
5. On success, the transaction commits and the cart is cleared; the interface shows confirmation with the order identifier and total.
6. On failure, the transaction rolls back so there is no partial Order, no partial OrderItem set, and the cart remains intact.

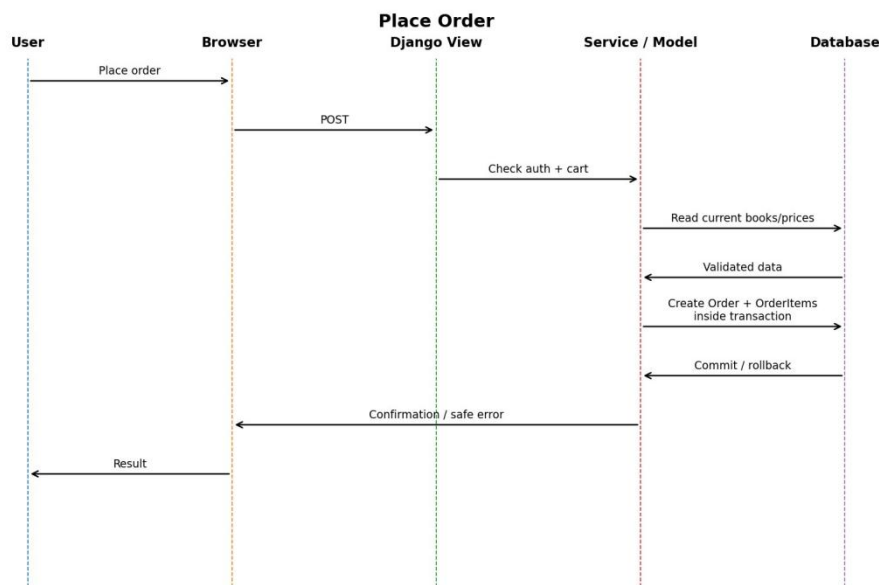


Figure 6. Place-order interaction sequence.

Placing an order only records the order; no payment is processed (BR-07).

4.3 API Design

The SRS requires browser-based interfaces and does not require a public REST API. The routes below are the Django browser routes of the design. Route names may be adjusted at implementation, but access rules must remain consistent with the RBAC matrix; tests verify behaviour, not exact URL names.

Route	Method	Access	Purpose / behavior
/register/	GET, POST	Public	Display registration form; validate input (including Role) and create account.
/login/	GET, POST	Public	Display login form and establish session after valid credentials.
/logout/	POST	Authenticated	Destroy current session and redirect safely.
/books/	GET	Public / Authenticated	Browse available book listings (paginated).
/books/search/	GET	Public / Authenticated	Search with bounded query (100 characters) and pagination (20 per page).
/books/add/	GET, POST	Authenticated Seller	Validate and create a book listing, recording the Seller.
/cart/	GET	Authenticated Customer	Display current customer's cart only.

Route	Method	Access	Purpose / behavior
/cart/add/	POST	Authenticated Customer	Add a cart item or increase the existing line after validation.
/cart/remove/<id>/	POST	Authenticated Customer	Remove an item only if it belongs to the current customer's cart (otherwise 404).
/orders/place/	POST	Authenticated Customer	Validate cart, calculate total, create order records atomically, clear cart on success.
/orders/	GET	Authenticated Customer	Display only the current customer's order history.

Interface behavior

- Use GET to display/retrieve information and POST for state-changing actions.
- Use CSRF protection on POST forms (SEC-CTRL-12).
- Redirect after successful form submission to reduce accidental duplicate submissions.
- Display field-level validation errors without exposing internal details.
- Redirect unauthenticated users to login; deny logged-in users whose role does not permit the requested action with HTTP 403.
- Validate post-login next/return redirects so external destinations are not accepted (SEC-CTRL-20).

4.4 Error Handling, Logging and Monitoring

User-facing errors

- Invalid registration: show field errors and do not create an account.
- Invalid login: use a generic authentication failure message.
- Wrong role: deny the action (HTTP 403) and do not modify data.
- Missing book or invalid quantity: explain the validation issue and leave the cart unchanged.
- Empty cart: do not create an order; explain the required correction.
- Unexpected/database failure: show a generic message, log safe diagnostics, and never expose stack traces.

Logging and operational checks

- Log failed login, denied access and failed order events with user identifier and timestamp, without secrets or sensitive values (SEC-CTRL-18).
- Use separate development and hosted-environment settings; DEBUG must be disabled when hosted.
- Monitor application availability and application/database errors in the selected environment.
- Measure search latency against the three-second SRS target.
- Back up persistent data and periodically verify restore procedures.
- Run Django deployment checks (check --deploy) before demonstration/deployment.

4.5 UX Design

- Use consistent navigation, labels, page layout and feedback patterns; every operation ends with a visible success or error message.
- Make catalogue browsing and search easy to reach; each major operation is reachable within three clicks of the home page.
- Display book title, author, price and description where available.
- Separate seller listing actions from customer purchasing actions.

- Show cart contents (title, quantity, unit price, line total, cart total) and confirmation after adding/removing items.
- After order placement, show confirmation with the saved order identifier and total.
- Show clear empty states for no search matches, empty cart and no order history.
- Use readable text, labelled controls, keyboard-friendly navigation and responsive layouts.

4.6 Integration Contracts and Build Order

Integration means the four feature modules (accounts, books, cart, orders) work together without breaking each other's rules. The following contracts are adopted.

Boundary	Contract
accounts → all	Expose request.user and the server-side user.role, plus the role_required decorator. Other modules do not implement their own role logic.
books → cart	Cart references Book by FK only. Cart does not copy the Book price.
cart → orders	Order placement reads the user's cart through one service function (for example get_cart_items(user)) and clears it only after successful transaction completion.
orders → books	Order placement reads the current Book price and stores it in OrderItem.unit_price at purchase time.
all → templates	Use shared base templates for navigation, messages and safe error pages.

Integration/build sequence

1. Accounts: registration, login, logout, role storage and authentication. Validate SEC-CTRL-01 to SEC-CTRL-07.
2. Books: browse, search, seller add-listing. Validate the seller role gate on /books/add/.
3. Cart: add, remove, view. Validate ownership and quantity controls (SEC-CTRL-08, SEC-CTRL-10).
4. Orders: place order and history. Validate transaction, current-price calculation and cart clearing (SEC-CTRL-09, SEC-CTRL-11).
5. End-to-end: Seller lists a book → Customer finds it → Customer adds it → Customer orders it → Customer sees it in history; then run every denied cell of the RBAC matrix.

Integration test scenarios (identical to STP Section 16.4)

ID	Scenario	Expected result
IT-01	Seller adds a book, then a Customer searches for it.	Book appears with the correct stored price.
IT-02	A Customer adds a book to the cart; the stored price of that book is then changed through a maintenance or test-data update (Sellers cannot edit listings in this version); the Customer places the order.	The order total and OrderItem unit price use the current stored price (server-side).
IT-03	Customer places an order, then views history.	Order and items appear and the cart is empty.
IT-04	A failure is forced inside order creation.	No Order or OrderItems are committed and the cart remains intact.
IT-05	User A tries to remove/view User B's cart item or order.	Access is denied/404; no data changes and no ownership information is leaked.
IT-06	Session expires during checkout.	User is redirected to login and no partial order is created.

4.7 Resolved Decisions and Remaining Open Items

Item	Resolution	Where
Role selection at registration	Customer or Seller chosen at registration, validated by the server, not changeable.	FR-AUTH-01, BR-04
Seller purchasing	Customer-only; Sellers cannot purchase.	BR-03, RBAC matrix
Database engine	SQLite (development/demonstration); PostgreSQL if hosted.	SRS 2.1, 3.1.2
Security objectives	SO-1 to SO-6 approved.	SRS 3.5.1
Security control IDs	SEC-CTRL-01 to 20 retained, under SEC-01/SEC-02.	SRS 3.5.4
Book ownership	Book records its creating Seller; no edit/delete in this version.	FR-BOOK-03
Use-case diagram	Approved UML diagram is SRS Figure 1, reproduced in Section 3.11.	SRS 3.1.4
Login throttling	SEC-CTRL-16 is optional and outside the mandatory acceptance baseline.	SRS 3.5.4
Open: Deployment environment	Choose local demonstration, managed hosting, VPS/cloud VM or containerized deployment before the demonstration. No requirement depends on it.	SRS Appendix D
Open: Final RTM/code alignment	After implementation, replace remaining implementation placeholders with actual module, model, route and test references.	SRS Appendix C

5 Appendices

5.1 Glossary

Term	Definition
Authentication	Verifying a user's identity.
Authorization	Determining whether a user may perform an operation.
Anonymous Visitor	A user who is not logged in.
Customer	Registered user who browses and purchases books.
Seller	Registered user who adds book listings.
Book listing	Information representing a book available on the platform.
Cart item	A selected book and quantity associated with a customer.
Order	Persistent record of a customer's purchase.
Order item	A line in an order with a book, quantity and purchase-time unit price.
Django ORM	Django's model-based interface for database operations.
Layered modular monolith	One deployable application divided into modules/layers with clear responsibilities.
Traceability	Mapping a requirement to architecture, design, implementation and verification activities.
RBAC	Role-Based Access Control.
CSRF	Cross-Site Request Forgery.
XSS	Cross-Site Scripting.
IDOR	Insecure Direct Object Reference.

5.2 References

1. Online Bookstore Software Requirements Specification, Version 1.1 – Final, PES University, October 2026.
2. Online Bookstore Software Test Plan, Version 1.1, PES University, October 2026.
3. Online Bookstore – Security & Integration Plan (M4), Version 1.1, October 2026.
4. Software Architecture and Design Specification template supplied for the project.
5. Django documentation for authentication, password hashing, forms, models, transactions, CSRF protection, deployment checks and testing; consult the version selected for implementation.

5.3 Tools

- Microsoft Word or compatible DOCX editor.
- PlantUML or diagrams.net (draw.io) for editable UML diagrams.
- Django test framework and Python test tools.
- Git for version control and review.

Document Review Checklist

Check	Status
Every SRS requirement ID (FR-AUTH-01 to 03, FR-BOOK-01 to 03, FR-CART-01 to 03, FR-ORDER-01 to 02, NFR-01 to 07, SEC-01, SEC-02) appears in Section 3.9 with at least one STP test case.	Done
Route, component, module and role names are identical across SAD, SRS, STP and Security & Integration Plan.	Done
SRS SEC-01/SEC-02 are distinct from the detailed controls SEC-CTRL-01 to 20.	Done
Data model matches SRS Figure 2 and Appendix B.	Done
Every protected route matches the RBAC matrix and has positive/negative tests.	Done (tests in STP)
Use-case diagram uses the same roles and scope as the SRS.	Done
No proposed/TBD item remains except the open items in Section 4.7.	Done
Final code, models and routes match this document.	Pending – after implementation
Approvals recorded.	Pending – signatures